

Databases and SQL

SQL 4 -- Joins

Presented By: Rehber Moin

Agenda

- Recap: Cross Product / Cartesian Product
 - Intro to Joins
 - Inner Join
 - Outer Joins
 - Left Join
 - Right Join
 - Self Join
 - Joining Multiple Tables
 - How to Approach Problems Involving Joins
-

Precap: Cross Product (Cartesian Product)

Before understanding Joins, recall the mathematical **Cross Product**:

Set $X = \{1, 2, 3\}$

Set $Y = \{A, B\}$

$X \times Y = \{(1,A), (1,B), (2,A), (2,B), (3,A), (3,B)\}$

Every element of X is paired with every element of Y . The result has $|X| * |Y|$ elements.

In SQL, this is called a **Cross Join** -- every row from Table 1 is paired with every row from Table 2.

Intro to Joins

The Problem

Consider our two tables:

Students:

id	name	psp	batch_id
1	Schumacher	87	1
2	Nikki	90	1
3	Verstappen	92	2
4	Hulkenberg	87	1
5	Charles	87	3

Batches:

id	name
1	LLD
2	SQL
3	HLD

"Give me the Name and PSP of every student." -- Easy, it's all in one table.

"Give me the Name and Batch of every student." -- This requires data from **two** tables. Is it even possible with a single query?

Yes -- using **Joins**.

Cross Join (What Happens Without a Condition)

```
SELECT * FROM students JOIN batches;
```

This produces a **Cross Join** -- every student row is paired with every batch row:

students				batches			
id	name	psp	batch_id	id	name		
1	Schumacher	87	1	1	LLD	<-- makes sense	
1	Schumacher	87	1	2	SQL	<-- doesn't make sense	
1	Schumacher	87	1	3	HLD	<-- doesn't make sense	
2	Nikki	90	1	1	LLD		
2	Nikki	90	1	2	SQL		
... (5 students x 3 batches = 15 rows total)							

Most of these rows are **nonsensical**. To fix this, we need to specify **which column to join on**.

Complexity of a Join

Q: If Table T1 has n rows and Table T2 has m rows, what is the complexity of a join?

A: $O(n \times m)$ -- in the worst case (cross join), every row of T1 is compared with every row of T2.

Inner Join

An **Inner Join** returns only the rows where the join condition is met -- i.e., rows that have **matching values** in both tables.

```
SELECT * FROM students
JOIN batches
ON students.batch_id = batches.id;
```

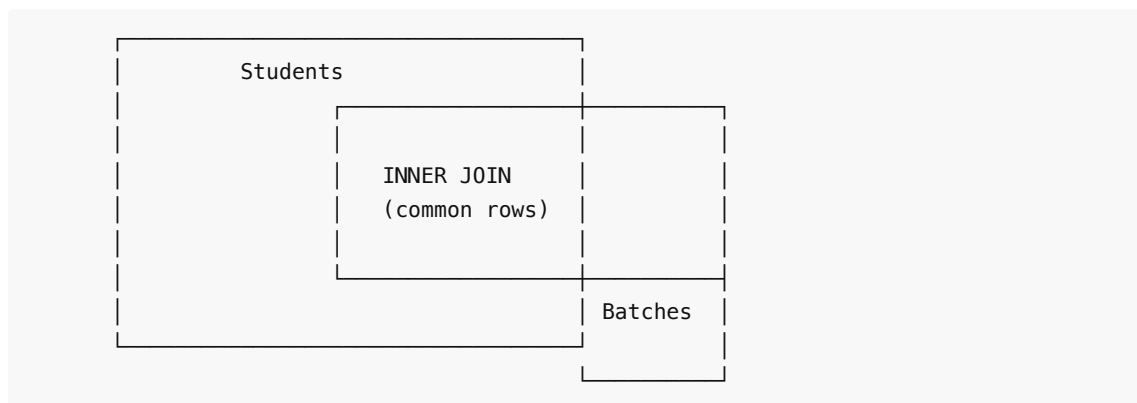
By default, `JOIN` in MySQL is an **Inner Join**. You can optionally write `INNER JOIN` for clarity.

How It Works

students				batches		
id	name	psp	b_id		id	name
1	Schumacher	87	1	→	1	LLD
2	Nikki	90	1	→	1	LLD
3	Verstappen	92	2	→	2	SQL
4	Hulkenberg	87	1	→	1	LLD
5	Charles	87	3	→	3	HLD

Result: All 5 students appear because every student has a valid `batch_id` that exists in `batches`.

Venn Diagram



Inner Join returns only the **intersection** -- rows where both tables have matching data.

Sakila Example: Film Names with Language

```
USE sakila;

SELECT film.title AS 'Name', language.name AS 'Language'
FROM film
JOIN language
ON film.language_id = language.language_id;
```

Outer Joins

Outer Joins return all rows from one (or both) tables, even if there is **no match** in the other table. Unmatched columns are filled with `NULL`.

Left Join

Give me **all records from the left table** regardless of whether there is a corresponding row in the right table.

Setup

```
-- Add a student with a batch_id that doesn't exist in batches
INSERT INTO students (name, psp, batch_id)
VALUES ('Nikita', 88, 8);
```

Now Nikita has batch_id = 8 , but there is no batch with id = 8 .

Query

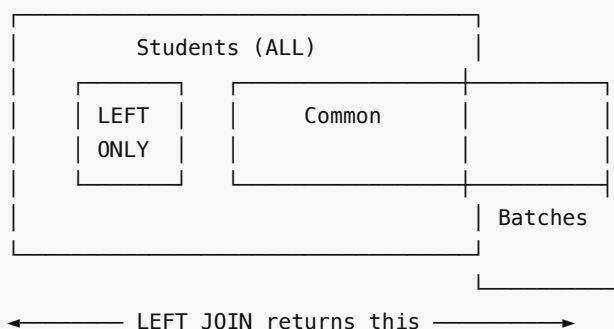
```
SELECT * FROM students
LEFT JOIN batches
ON students.batch_id = batches.id;
```

Result

id	name	psp	b_id	b.id	b.nm	
1	Schumacher	87	1	1	LLD	Match
2	Nikki	90	1	1	LLD	Match
3	Verstappen	92	2	2	SQL	Match
4	Hulkenberg	87	1	1	LLD	Match
5	Charles	87	3	3	HLD	Match
6	Nikita	88	8	NULL	NULL	No match (LEFT JOIN keeps it)

Nikita appears even though her batch_id = 8 has no match. The batch columns become NULL .

Venn Diagram



Right Join

Give me **all records from the right table** regardless of whether there is a corresponding row in the left table.

Setup

```
-- Add batches that no student is enrolled in
INSERT INTO batches (name)
VALUES ('DSA'), ('AI ML');
```

Query

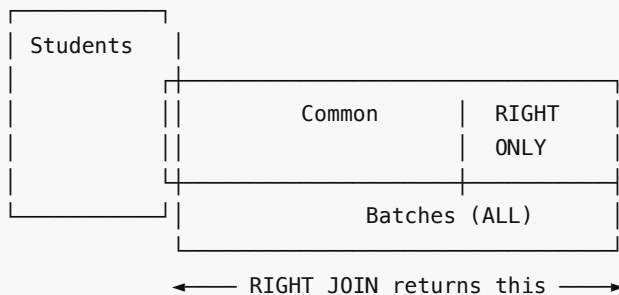
```
SELECT * FROM students
RIGHT JOIN batches
ON students.batch_id = batches.id;
```

Result

s.id	s.name	psp	b_id	id	name	
1	Schumacher	87	1	1	LLD	Match
2	Nikki	90	1	1	LLD	Match
4	Hulkenberg	87	1	1	LLD	Match
3	Verstappen	92	2	2	SQL	Match
5	Charles	87	3	3	HLD	Match
NULL	NULL	NULL	NULL	4	DSA	No match (RIGHT JOIN keeps it)
NULL	NULL	NULL	NULL	5	AI ML	No match (RIGHT JOIN keeps it)

DSA and AI ML appear even though no students are enrolled. The student columns become **NULL**.

Venn Diagram



Left Join = Right Join (Swapped Tables)

Whatever you can do with a Left Join, you can do with a Right Join by swapping the table order:

```
-- These two produce the same result:
SELECT * FROM students LEFT JOIN batches ON students.batch_id = batches.id;
SELECT * FROM batches RIGHT JOIN students ON batches.id = students.batch_id;
```

Joins with WHERE and ORDER BY

You can combine Joins with `WHERE` and `ORDER BY` just like any other query:

```
-- Students with their batches, only where batch exists, sorted by name
SELECT * FROM students
JOIN batches ON students.batch_id = batches.id
WHERE batches.id IS NOT NULL
ORDER BY students.name;

-- Students with 'u' in their name, joined with their batch
SELECT * FROM students
JOIN batches ON students.batch_id = batches.id
WHERE students.name LIKE '%u%';
```

ON vs WHERE for Filtering (Important Distinction)

Inner Join: ON and WHERE behave the same for filters

```
-- Filter in WHERE (standard)
SELECT * FROM students
JOIN batches ON students.batch_id = batches.id
WHERE students.name LIKE '%u%';

-- Filter in ON (same result for INNER JOIN)
SELECT * FROM students
JOIN batches ON students.batch_id = batches.id AND students.name LIKE '%u%';
```

Both return the same result because Inner Join only keeps matching rows anyway.

Left/Right Join: ON and WHERE behave DIFFERENTLY

```
-- Filter in ON clause (LEFT JOIN)
SELECT * FROM students
LEFT JOIN batches
ON students.batch_id = batches.id AND students.name LIKE '%u%';
```

What happens: The `ON` condition only determines whether a row from the **right table** (batches) matches. It does **NOT** remove rows from the left table (students). If the `ON` condition fails, the student still appears, but the batch columns become `NULL`.

```
-- Filter in WHERE clause (LEFT JOIN)
SELECT * FROM students
LEFT JOIN batches ON students.batch_id = batches.id
WHERE students.name LIKE '%u%';
```

What happens: The `WHERE` filters **after** the join. Students without 'u' in their name are removed entirely from the result.

Rule of thumb:

- Use `ON` for the **join condition** (how tables relate).
- Use `WHERE` for **filtering** the final results.
- For Outer Joins, putting filters in `ON` vs `WHERE` gives **different results**.

Self Joins

A **Self Join** is when a table is joined with **itself**. This is useful when rows in a table have relationships with other rows in the **same** table.

Cross Product Reminder

Set $A = \{1, 2\}$

$A \times A = \{(1,1), (1,2), (2,1), (2,2)\}$

Example: Employee-Manager Hierarchy

```
CREATE TABLE employee (  
  id          INT PRIMARY KEY AUTO_INCREMENT,  
  name        VARCHAR(50),  
  manager_id  INT  
);  
  
INSERT INTO employee (name) VALUES ('Tony');  
INSERT INTO employee (name, manager_id)  
VALUES ('Steve', 1), ('Natasha', 1), ('Peter', 2), ('Thanos', 2);
```

Employee Table:

id	name	manager_id
1	Tony	NULL
2	Steve	1
3	Natasha	1
4	Peter	2
5	Thanos	2

Q: Is a manager also an employee of the company? **Yes!** That's why we can self-join.

Hierarchy Diagram

```
      Tony (id=1, manager=NULL)  
     /  \  
  Steve  Natasha  
(id=2)  (id=3)
```

/	\
Peter	Thanos
(id=4)	(id=5)

Self Join: Employee with Manager Name

We join the table with itself using **aliases** (e1 for the employee, e2 for the manager):

```
SELECT e1.name AS 'Employee', e2.name AS 'Manager'
FROM employee e1
JOIN employee e2
ON e1.manager_id = e2.id;
```

Result:

Employee	Manager
Steve	Tony
Natasha	Tony
Peter	Steve
Thanos	Steve

Tony is **missing** because he has no manager (`manager_id = NULL`) and the Inner Join excludes him.

Self Join with LEFT JOIN (Include Employees Without Managers)

```
SELECT e1.name AS 'Employee', e2.name AS 'Manager'
FROM employee e1
LEFT JOIN employee e2
ON e1.manager_id = e2.id;
```

Result:

Employee	Manager
Tony	NULL
Steve	Tony
Natasha	Tony
Peter	Steve
Thanos	Steve

Now Tony appears with `NULL` as his manager.

Self Join with RIGHT JOIN


```
SELECT e1.name AS 'Employee', e2.name AS 'Manager'
FROM employee e1
RIGHT JOIN employee e2
ON e1.manager_id = e2.id;
```

Result:

Employee	Manager
Steve	Tony
Natasha	Tony
Peter	Steve
Thanos	Steve
NULL	Natasha
NULL	Peter
NULL	Thanos

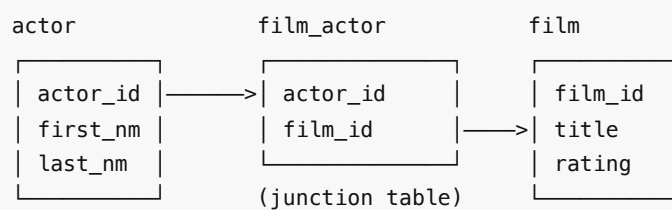
Natasha, Peter, and Thanos appear on the right (as managers) with **NULL** employees -- because nobody reports to them.

Joining Multiple Tables

Sometimes data spans **more than two tables**. You can chain multiple Joins.

Example: Actors and Their Films (Sakila)

The relationship goes through a **junction table** (`film_actor`):



```
USE sakila;

SELECT
    CONCAT(actor.first_name, ' ', actor.last_name) AS 'Actor Name',
    film.title AS 'Film Name'
FROM actor
JOIN film_actor ON actor.actor_id = film_actor.actor_id
JOIN film       ON film_actor.film_id = film.film_id
ORDER BY actor.first_name;
```

Assignment: Find Actors in 'DINOSAUR SECRETARY'

```
SELECT
    CONCAT(actor.first_name, ' ', actor.last_name) AS 'Actor Name',
    film.title AS 'Film Name'
FROM actor
JOIN film_actor ON actor.actor_id = film_actor.actor_id
JOIN film      ON film_actor.film_id = film.film_id
WHERE film.title = 'DINOSAUR SECRETARY'
ORDER BY actor.first_name;
```

Assignment: All Actors in PG-13 Films

```
SELECT
    CONCAT(actor.first_name, ' ', actor.last_name) AS 'Actor Name',
    film.title AS 'Film Name'
FROM actor
JOIN film_actor ON actor.actor_id = film_actor.actor_id
JOIN film      ON film.film_id = film_actor.film_id
WHERE film.rating = 'PG-13';
```

How to Approach Problems Involving Joins

A step-by-step framework:

```
Step 1: Identify the Tables involved
      |
Step 2: Write the SELECT statement (leave columns blank for now)
      |
Step 3: Perform the JOIN(s) with ON conditions
      |
Step 4: Add WHERE clause for filtering
      |
Step 5: Add ORDER BY for sorting
      |
Step 6: Now go back and fill in the SELECT columns
```

Why This Order?

- You need to know **which tables** before you can join them.
- You need to **join** before you know which columns are available.
- **Filtering** and **sorting** come after the data is assembled.
- Writing the `SELECT` columns **last** avoids going back to fix alias/table references.

Example Walkthrough

Problem: Find the names of actors who acted in PG-13 rated films, along with the film names.

```

-- Step 1: Tables needed: actor, film_actor, film
-- Step 2: SELECT ... FROM actor
-- Step 3: JOIN film_actor ON actor.actor_id = film_actor.actor_id
--          JOIN film ON film_actor.film_id = film.film_id
-- Step 4: WHERE film.rating = 'PG-13'
-- Step 5: ORDER BY actor.first_name (optional)
-- Step 6: SELECT CONCAT(actor.first_name, ' ', actor.last_name), film.title

-- Final query:
SELECT
    CONCAT(actor.first_name, ' ', actor.last_name) AS 'Actor Name',
    film.title AS 'Film Name'
FROM actor
JOIN film_actor ON actor.actor_id = film_actor.actor_id
JOIN film       ON film_actor.film_id = film.film_id
WHERE film.rating = 'PG-13'
ORDER BY actor.first_name;

```

Summary

Join Type	Returns
Cross Join	Every row of T1 paired with every row of T2. No condition. $O(n \times m)$.
Inner Join	Only rows where the join condition matches in both tables.
Left Join	All rows from the left table + matched rows from the right (NULL if no match).
Right Join	All rows from the right table + matched rows from the left (NULL if no match).
Self Join	A table joined with itself (uses aliases to distinguish the two copies).

Concept	Key Takeaway
ON vs WHERE (Outer Join)	ON determines matches; WHERE filters after. Different results!
Left = Right (swapped)	A LEFT JOIN B is the same as B RIGHT JOIN A.
Multiple Joins	Chain JOINS for data across 3+ tables (use junction tables).
Approach framework	Tables -> SELECT -> JOIN -> WHERE -> ORDER BY -> Columns.